Arithmetic Logic Unit (ALU)
MULTIPLICATION OF NUMBERS

Sem: 4th Section: COA-CSE-G1 & COA-CSCE-G1 Faculty: Prof. Anil Kumar Swain

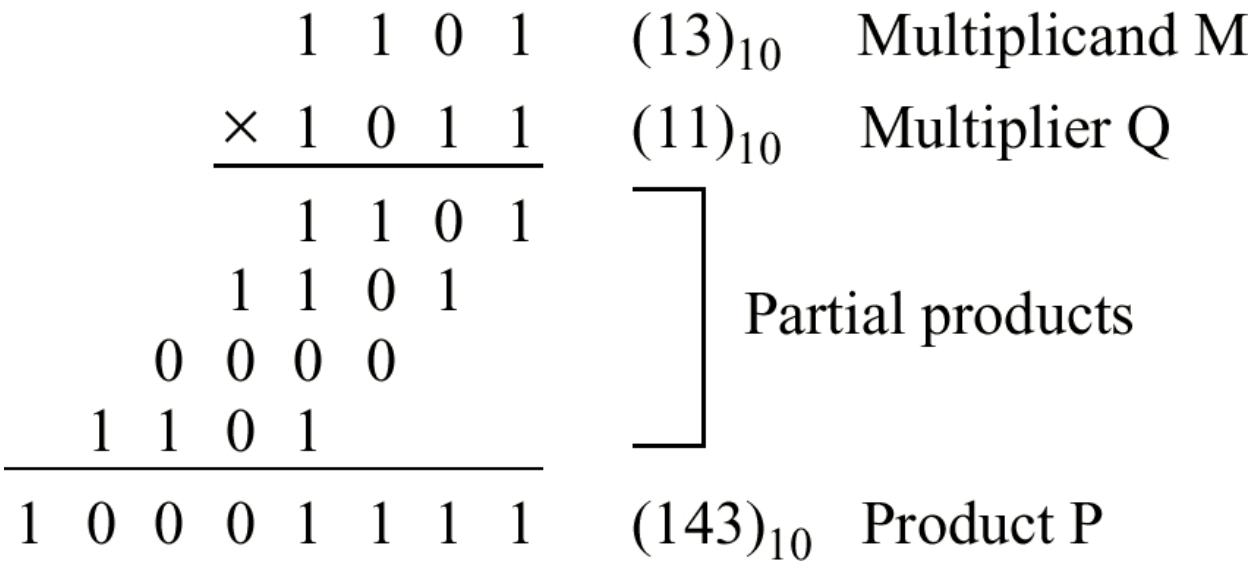
Dt.25.03.2020

Lecture Summary

1. Multiplication of Unsigned Numbers
   - Array multiplier
   - Sequential multiplier
2. Multiplication of Signed Numbers
   - Booth Algorithms
3. Fast Multiplication
   - Bit-pair Recording of Multipliers
4. Home Assignments

1 Multiplication of Unsigned Numbers

- Compared to add/subtract operation, the algorithm for multiplication is much more complex needing substantially more hardware for its execution.
- The hand computation for binary multiplication is a repetitive process of left shift and add operation. Starting from the LSB of the multiplier, the multiplicand is copied as the first partial product if LSB is '1', else all '0's forms the first partial product. Subsequently, each of gthe high order multiplier bits is checked one at a time.
- The usual algorithm for multiplying integers by hand for the binary system is illustrated below.



- The product of two n-digit numbers can be accommodated in 2n digits. So Multiplication of two 4-bit unsigned binary integers produces an 8-bit result.

1.1 Method I – Using combinational logic circuit (Array Multipliers)

- The manual scheme of multiplication of positive numbers can be hardwired with a combinational array logic after necessary modification.
- Instead of storing the partial products and adding them at the end (in case of hand computation), the two dimensional logic can be so organized that the partial product at $(i+1)_{th}$ stage is the sum of the partial product of i_{th} stage and the left shifted multiplicand.
- Thus for a multiplicand $M = m_{n-1}\ m_{n-2}\ m_{n-3}\ldots\ m_1\ m_0$ and multiplier $Q = q_{n-1}\ q_{n-2}\ q_{n-3}\ldots\ q_1\ q_0$

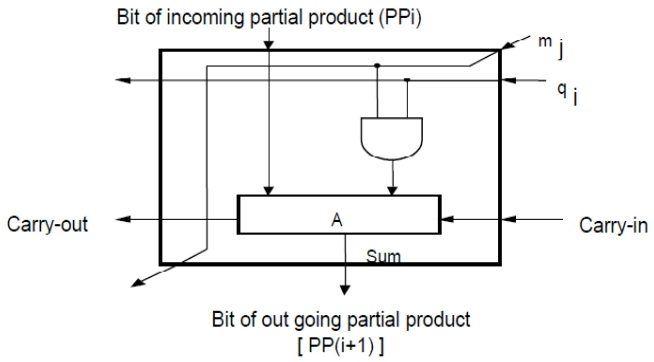
$$P = \sum_{i=0}^{n-1} q_i\, 2^i\, M = \sum_{i=0}^{n-1} 2^i \sum_{j=0}^{n-1} q_i\, m_j\, 2^j$$

- Denoting the ith partial product as PP_i

$PP_0 = 0$
$PP_1 = PP_0 + q_0\, M$
$PP_2 = PP_1 + q_1\ 2^1\, M$
$PP_3 = PP_2 + q_2\, 2^2\, M$
……..

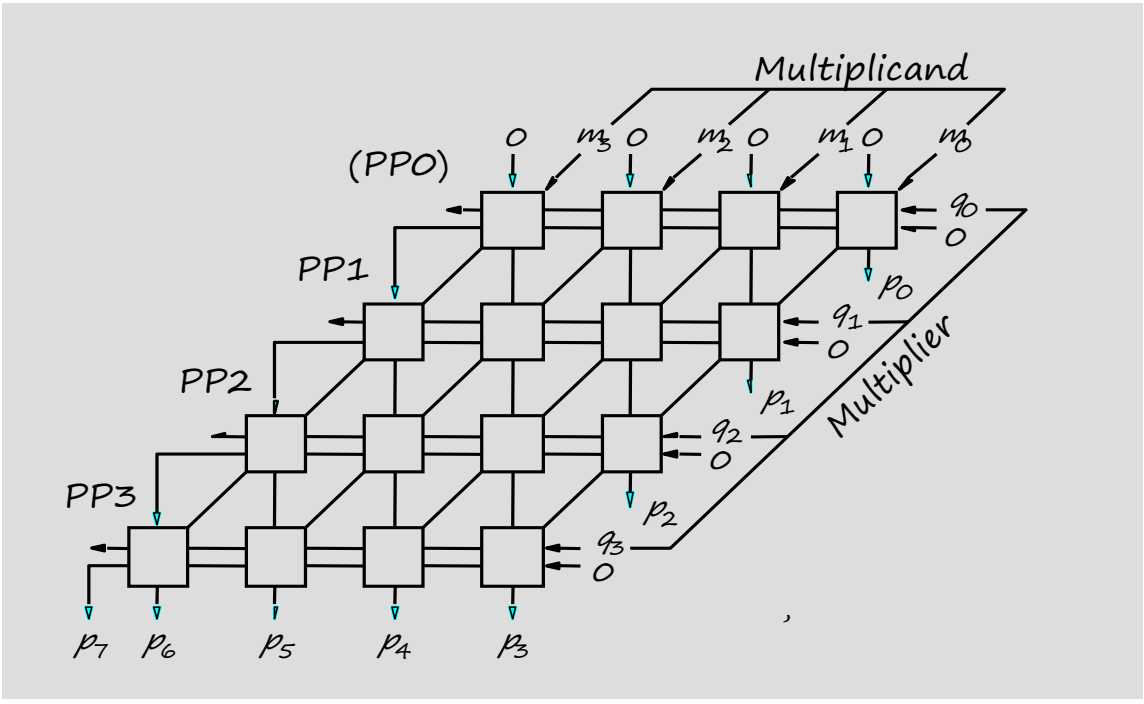
$PP_{i+1} = PP_i + q_i\, 2^i\, M$
……..

$P = PP_n = PP_{n-1} + q_{n-1}\, 2^{n-1}\, M$



(Typical Cell of Multiplication Array Implementation)

- The basic combinational cell used in the array as the building block to handle one bit of the partial product as shown below.The main component in any cell is an Full Adder circuit. The AND gate in any cell determines whether the multiplicand bit mj, is added to the partial product bit, based on the value of the multiplier bit qi. The multiplicand is shifted left one position per row by the diagonal signal path.



- **Rules to implement multiplication are:**
  - If the i^{th} bit of the multiplier is 1, shift the multiplicand and add the shifted multiplicand to the current value of the partial product.
  - Hand over the partial product to the next stage.
  - Value of the partial product at the start stage is 0.

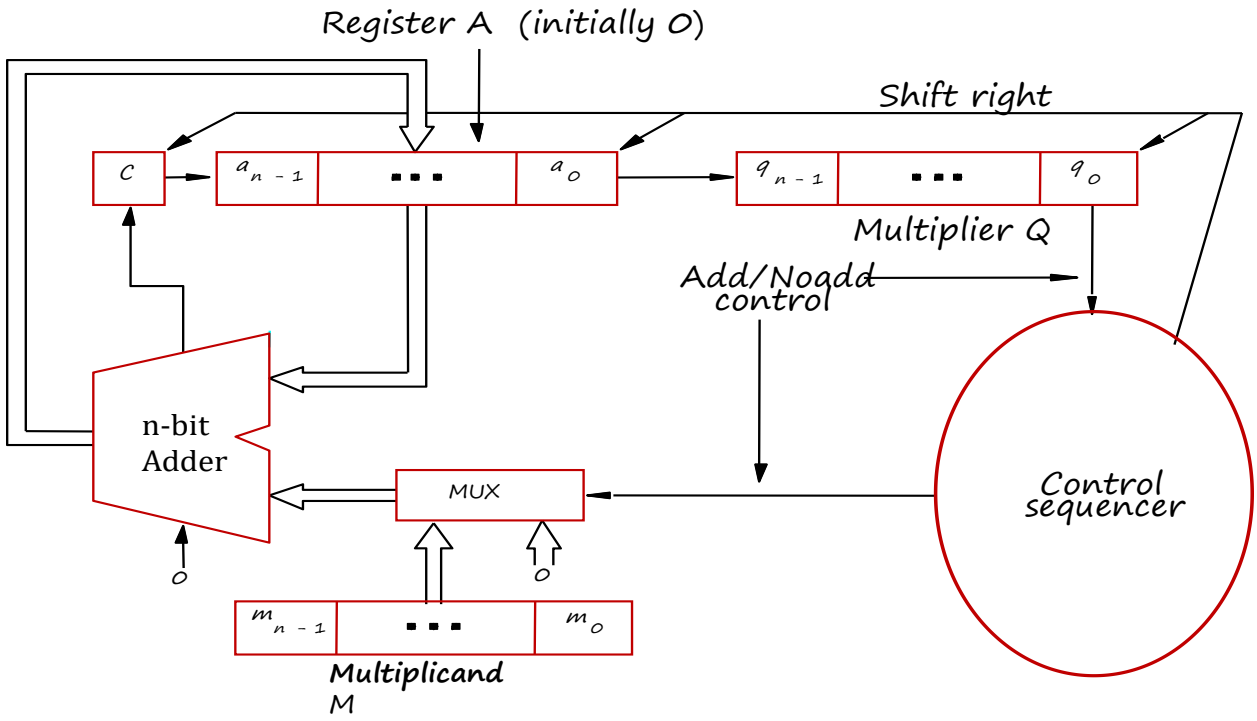
- Advantages:
  - Minimum complexity
  - Easily scalable
  - Easily pipelined
  - Regular shape, easy to place & route

- Disadvantages:

  Array multipliers are **highly inefficient**:
  – Need n n-bit adders => number of gate counts is proportional to n^2
  – Impractical for large numbers such as 32-bit or 64-bit numbers typically used in computers
  – Perform only one function, namely, unsigned integer product.

1.2 Method II – Using combinational logic circuit & sequential techniques (Sequential Multiplier)

- Multiplication can also be performed using a mixture of combinational array techniques as shown above and sequential techniques. The simplest hardware arrangement for sequential multiplication is as shown.



(Sequential Circuit Multiplier)

- **Control logic and registers**
  - n bit registers, 1 bit carry register C
  - Register set up
    - Q register ← multiplier
    - M register ← multiplicand
    - A register ← 0
    - C ← 0
  - C for carries after addition
  - Product will be 2n bits in A Q registers
- The circuit performs the multiplication by using **a single n-bit adder n time**. Register A and Q combined to hold the Partial Product i (PPi) while the multiplier bit qi generates the signal Add/Noadd. This signal controls the addition of multiplicand M to PPi to generate PP(i+1). The product is computed in n cycles. The partial product grows in length by one bit in each cycle. The carry-out from the adder is stored in flip flop C.
- **Key Observation:**
  - Adding a left-shifted multiplicand to an unshifted partial product is equivalent to adding an unshifted multiplicand to right-shifted partial product.
- **Algorithm for unsigned binary multiplication (Sequential Multiplier Algorithm)**

  Step-1: Initialize C=A=0, Q=multiplier value, M=multiplicand value

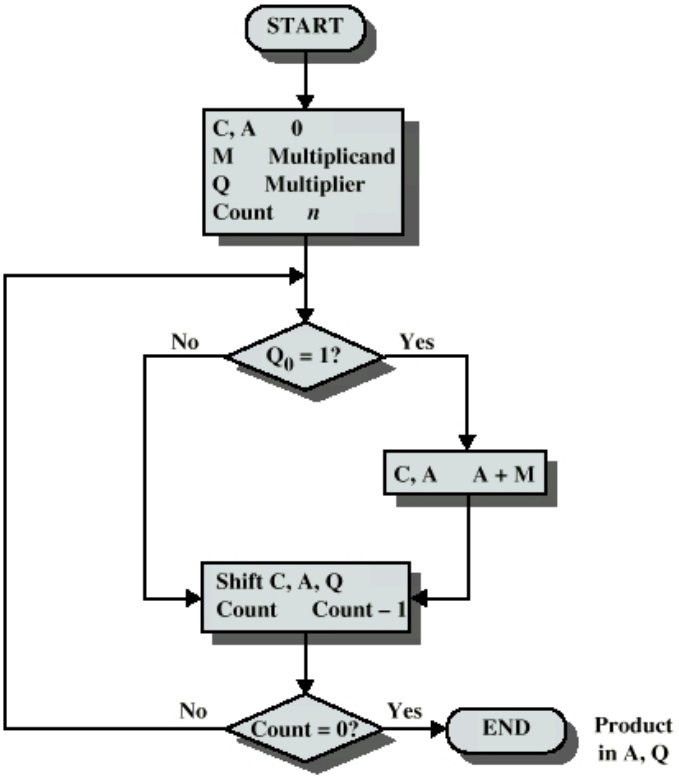
  Step-2: Do the following n times

  i) If $q_0 = 1$, Add M into A (Perform A = A + M), store carry in C,
  If $q_0 = 0$, No-Add (Donot do anything)

  ii) Shift C, A, Q right one bit so that
  $a_{n-1} \leftarrow C$ (The carry flag bit will be shifted to MSB of A)
  $q_{n-1} \leftarrow a_0$ (The LSB of A will be shifted to MSB of Q)
  q_0 is lost

  Step-3: Product is in A, Q

- **<u>Flow Chart for unsigned binary multiplication</u>**



- **Examples on unsigned binary multiplication**

Example-1: Multiply the following pairs of unsigned numbers using combinational logic circuit & sequential hardware (sequential multiplier algorithm).

X = 1101 Y = 1011

Assume X is the multiplicand and Y is the multiplier

Soln:

Carry	Register	Multiplier Register	Multiplicand Register	Operation	Remark
C	A	Q	M		
0	0 0 0 0	1 0 1 1	1 1 0 1	Initial configuration	
0	1 1 0 1	1 0 1 1		Add (A = A + M)	First Cycle
0	0 1 1 0	1 1 0 1		Shift C, A, Q right	
1	0 0 1 1	1 1 0 1		Add (A = A + M)	Second Cycle
0	1 0 0 1	1 1 1 0		Shift C, A, Q right	
0	1 0 0 1	1 1 1 0		No-Add	Third Cycle
0	0 1 0 0	1 1 1 1		Shift C, A, Q right	
1	0 0 0 1	1 1 1 1		Add (A = A + M)	Fourth Cycle
0	1 0 0 0	1 1 1 1		Shift C, A, Q right	

Example-2: Perform the following positive number multiplication

22 X 5

by using sequential circuit multiplier algorithm.

Soln:

22 in binary is 10110 and 5 in binary is 00101

Carry	Register	Multiplier Register	Multiplicand Register	Operation	Remark
C	A	Q	M		
0	00000	00101	10110	Initial configuration	
0	10110	00101		Add (A = A + M)	First Cycle
0	01011	00010		Shift C, A, Q right	
0	01011	00010		No-Add	Second Cycle
0	00101	10001		Shift C, A, Q right	
0	11011	10001		Add (A = A + M)	Third Cycle
0	01101	11000		Shift C, A, Q right	
0	01101	11000		No-Add	Fourth Cycle
0	00110	111100		Shift C, A, Q right	
0	00110	11100		No-Add	Fifth Cycle
0	00011	01110		Shift C, A, Q right	

2 Multiplication of Signed Numbers (Signed Operand Multiplication)

- Multiplication of signed operands in 2's-compliment system generates a double length product. The general strategy is to accumulate partial products.
- **Case I : <u>Positive Multiplier</u> with Negative Multiplicand (or Positive Multiplicand)**
  - We can not directly apply the rule of hand/unsigned multiplication to a signed multiplication having positive multiplier and negative multiplicand.
  - First perform the multiplication like hand multiplication, then **extend the sign-bit** value of the multiplicand to the left as far as the product will extend.
- Example
  Let Multiplicand = M = -12 and Multiplier = Q = +5. Perform the signed multiplication.
  Soln:
  In sign-2's complement form M = 10100 and Q = 00101

Case I.a : <u>Positive Multiplier</u> with Negative Multiplicand

```
                1  0  1  0  0   (-12)
             x  0  0  1  0  1   (+5)
             ------------------
                1  0  1  0  0
             0  0  0  0  0
          1  0  1  0  0
       0  0  0  0  0
    0  0  0  0  0
    ------------------------
    0  0  1  1  0  0  1  0  0
```

As the sign. bit of product is zero, so the product is +ve, so,
This product in decimal is = $2^6 + 2^5 + 2^2$
= 64 + 32 + 4 = 100
This **product is wrong** as the result should be
-12 X +5 = -60
In order to get correct result, extend the sign bit in the partial product as shown above

```
                1  0  1  0  0   (-12)
             x  0  0  1  0  1   (+5)
             ------------------
          1  1  1  1  1  1  0  1  0  0
          0  0  0  0  0  0  0  0  0
          1  1  1  1  0  1  0  0
          0  0  0  0  0  0  0
          0  0  0  0  0  0
          ------------------------
       1  1  1  1  1  0  0  0  1  0  0
```

Discard it
Sign. Bit.
As the sign. of the product is 1, so the product is –ve (is in sign-2's complement form). So the product in decimal is
= - (2's of the product)
= - (2's of 1111000100)
= - (0000111100) = - ($2^5 + 2^4 + 2^3 + 2^2$)
= - (32+16+8+4) = -60 (**correct result**)

Case I.b : <u>Positive Multiplier</u> with Positive Multiplicand

<u>Example-1</u>
M = 01100 (+12) Q = 00101 (+5)

```
                0  1  1  0  0   (+12)
             x  0  0  1  0  1   (+5)
             ------------------
          0  1  1  0  0  0  0  0  0  0
          0  0  0  0  0  0  0  0  0
          0  0  0  0  1  1  0  0
          0  0  0  0  0  0  0
          0  0  0  0  0  0
          ------------------------
       0  0  0  0  1  1  1  1  0  0
```

Sign. Bit.
As the sign. of the product is 0, so the product is +ve.
So the product in decimal is
= 0000111100 = $2^5 + 2^4 + 2^3 + 2^2$
= 32+16+8+4) = +60

<u>Example-2</u>
M = 01101 (+13) Q = 01011 (+11)

Solve it by Case-I method.

- **Case II : <u>Negative Multiplier</u> with Positive Multiplicand (or Negative Multiplicand)**
  For a negative multiplier, solution is to form 2's-compliment of both the multiplier and the multiplicand and proceed as earlier. This is because complementation of both operands does not change the value or sign of the product.

Case II.a : <u>Negative Multiplier</u> with Positive Multiplicand

<u>Example-1</u>

Given Multiplicand = M = 01100 (+12)
Multiplier = Q = 11011 (-5)

Solution:

As the Multiplier is –ve, so it is case-II. First find out the 2's of both the Multiplicand and Multiplier, then perform the multiplication as per case-I method.

M^{2s} =10100 Q^{2s} = 00101

```
                          1  0  1  0  0   (-12)
                    x     0  0  1  0  1   (+5)
                  ---------------------
      1  1  1  1  1  1  0  1  0  0
      0  0  0  0  0  0  0  0  0
      1  1  1  1  0  1  0  0
      0  0  0  0  0  0  0
      0  0  0  0  0  0
   ------------------------------------
1     1  1  1  1  0  0  0  1  0  0
```

Discard it Sign. bit.

As the sign. of the product is 1, so the product is –ve (is in sign-2's complement form). So the product in decimal is

= - (2's of the product)
= - (2's of 1111000100)
= - (0000111100) = - ($2^5 + 2^4 + 2^3 + 2^2$)
= - (32+16+8+4) = -60 (**correct result**)

<u>Example-2</u>

Given Multiplicand = M = 01101 (+13)
Multiplier = Q = 10101 (-11)

Perform the Multiplication.

Solution:

Do it yourself.

Case II.b : Negative Multiplier with Negative Multiplicand

Example-1

Given Multiplicand = M = 10100 (-12)
Multiplier = Q = 11011 (-5)

Solution:

As the Multiplier is –ve, so it is case-II. First find out the 2's of both the Multiplicand and Multiplier, then perform the multiplication as per case-I method.

M^{2s} =01100 Q^{2s} = 00101

					0	1	1	0	0	(+12)
				x	0	0	1	0	1	(+5)
0	**0**	**0**	**0**	**0**	0	1	1	0	0	
0	**0**	**0**	**0**	0	0	0	0	0		
0	**0**	**0**	0	1	1	0	0			
0	**0**	0	0	0	0	0				
0	0	0	0	0	0					
0	0	0	0	1	1	1	1	0	0	

Sign. Bit.

As the sign. of the product is 0, so the product is +ve.
So the product in decimal is
= 0000111100 = $2^5 + 2^4 + 2^3 + 2^2$
= 32+16+8+4) = +60

Example-2

Given Multiplicand = M = 10011 (-13)
Multiplier = Q = 10101 (-11)

Perform the Multiplication.

Solution:

Do it yourself.

3 BOOTH ALGORITHM

- For +ve and –ve multiplier, the method of signed-operand multiplication are different as discussed above. **A technique which is suitable for both negative and positive multipliers is called the Booth Algorithm.**
- It generates 2*n*-bit product and powerful algorithm for signed number multiplication.
- **Advantages**
  - It handles both positive and negative multipliers uniformly
  - It achieves some efficiency in the number of additions required when the multiplier has a few large blocks of 1's.
- **Booth encoding**
  - Apply encoding to the multiplier bits before the bits are used for getting partial products, that is to find out the **booth recording of a multiplier** first, then multiply this to multiplicand. In this scheme, −1 times the shifted multiplicand is selected when moving from 0 to 1 and +1 times the shifted multiplicand is selected when moving from 1 to 0, as the multiplier is scanned from right to left.
  - If the least significant bit is 0 or 1, then we must assume that an implied 0 lies to its right.
  - **Rule:**
    a) If i^{th} bit b_i is 0 and $(i-1)^{th}$ bit b_{i-1} is 1, then take b_i as +1
    b) If i^{th} bit b_i is 1 and $(i-1)^{th}$ bit b_{i-1} is 0, then take b_i as –1
    c) If i^{th} bit b_i is 0 and $(i-1)^{th}$ bit b_{i-1} is 0,then take b_i as 0
    d) If i^{th} bit b_i is 1 and $(i-1)^{th}$ bit b_{i-1} is 1, then take b_i as 0

Given Multiplier		Version of multiplicand selected by Bit b_i
Bit b_i	Bit b_{i-1}	
0	0	0 x M
0	1	+1 x M
1	0	-1 x M
1	1	0 x M

Remember: same same zero, different one with bit i as sign.

- **Examples of Booth Recording of a multiplier**
  Ex-1: Find out booth recording of a multiplier 0 0 1 1 1 1 0.

  Ans: 0 0 1 1 1 1 0 (0) ←-Assume

  0 +1 0 0 0 −1 0

  Ex-2:

Sl.	Given Multiplier	Booth recording of the multiplier
1	0 0 1 0 1 1 0 0 1 1 1 0 1 0 1 1 0 0	0 +1 -1 +1 0 -1 0 +1 0 0 -1 +1 -1 +1 0 1 0 0
2	1 1 0 1 0	0 −1 +1 −1 0
3	0 1 1 1 0 0 0 0	+1 0 0 –1 0 0 0 0
4	0 1 1 1 0 1 1 0	+1 0 0 –1 +1 0 –1 0
5	0 0 0 0 0 1 1 1	0 0 0 0 +1 0 0 –1
6	0 1 0 1 0 1 0 1	+1–1+1–1+1–1+1–1

- The transformation from original multiplier to booth recoded multiplier is called skipping over 1's
- **Example**
  Multiply +7 into -3. Use Booth's Algorithm.

Multiplicand, M=+7=0111 So 2' of M = M^{2s}=1001
Multiplier, Q=-3=1101 => Booth's Recording of Multiplier=0 -1 +1 -1

					0	1	1	1	M = (+7)	Remarks
				x	0	-1	+1	-1	Q = (-3)	
	1	**1**	**1**	**1**	1	0	0	1		-1xM = M^{2s}
	0	**0**	**0**	0	1	1	1			+1xM = M
	1	**1**	1	0	0	1				-1xM = M^{2s}
	0	0	0	0	0					0xM=0
1	1	1	1	0	1	0	1	1	←	Product

Discard it Sign. bit.

As the sign. of the product is 1, so the product is –ve (is in sign-2's complement form). So the product in decimal is
= - (2's of the product)
= - (2's of 11101011)
= - (00010101) = - ($2^4 + 2^2 + 2^0$)
= - (16+4+1) = -21 (**correct result**)

The worst case of an implementation using Booth's algorithm is when pairs of 01s or 10s occur very frequently in the multiplier

16-bit worst case multiplier

0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 (no blocks of 1's)

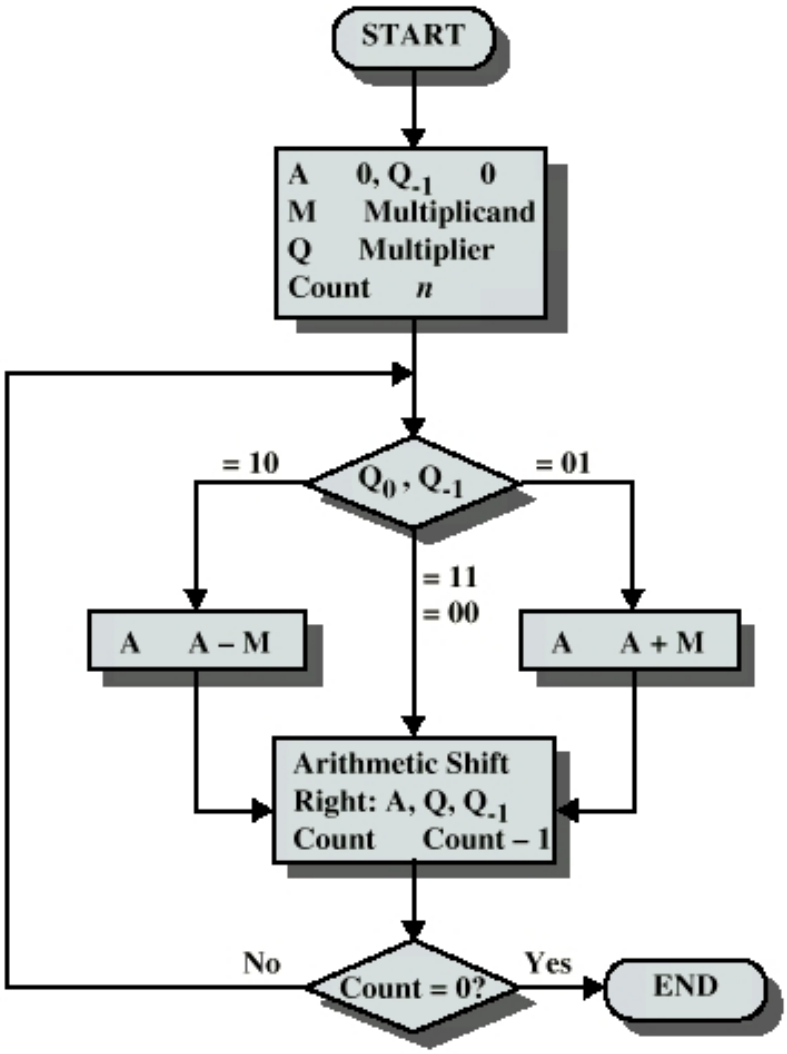
16-bit ordinary multiplier

1 1 0 0 0 1 0 1 1 0 1 1 1 1 0 0 (some blocks of 1's)

16-bit good multiplier

0 0 0 0 1 1 1 1 1 0 0 0 0 1 1 1 (large blocks of 1's)

- **Flow chart of Booth Algorithm.**



(Booth's Algorithm for 2's complementation Multiplication)

1. **Example of Booth's Algorithm (7 * 3 = +21)**
   Here Multiplicand, M=7=0111 and Multiplier, Q=3=0011
   2's complement of M, M^{2s}= 1001

Register	Multiplier Register		Multiplicand Register	Operation	Remark
A	**Q**	**Q_{-1}**	**M**		
0000	0011	0	0111	Initial configuration	
1001	0011	0		A=A-M=A+ M^{2s}	First Cycle
1100	1001	1		Shift A, Q, Q_{-1} right	
1100	1001	1		No Add/Sub	Second Cycle
1110	0100	1		Shift A, Q, Q_{-1} right	
0101	0100	1		A=A+M	Third Cycle
0010	1010	0		Shift A, Q, Q_{-1} right	
0010	1010	0		No Add/Sub	Fourth Cycle
0001	0101	0		Shift A, Q, Q_{-1} right	

Result = Contains of AQ register = 0001 0101 = 21

2. **Example of Booth's Algorithm (-7 * 3 = -21)**
   Here Multiplicand, M=-7= 1001 and Multiplier, Q=3=0011
   2's complement of M, M^{2s}= 0111

Register	Multiplier Register		Multiplicand Register	Operation	Remark
A	**Q**	**Q_{-1}**	**M**		
0000	0011	0	1001	Initial configuration	
0111	0011	0		A=A-M=A+ M^{2s}	First Cycle
0011	1001	1		Shift A, Q, Q_{-1} right	
0011	1001	1		No Add/Sub	Second Cycle
0001	1100	1		Shift A, Q, Q_{-1} right	
1010	1100	1		A=A+M	Third Cycle
1101	0110	0		Shift A, Q, Q_{-1} right	
1101	0110	0		No Add/Sub	Fourth Cycle
1110	1011	0		Shift A, Q, Q_{-1} right	

Result = Contains of AQ register = 1110 1011= -(2's of 1110 1011) =-(00010101) = -21

3. **Example of Booth's Algorithm (7 *-3 = -21)**
Here Multiplicand, M=7= 0111 and Multiplier, Q=-3=1101
2's complement of M, M^{2s}=1001

Register	Multiplier Register		Multiplicand Register	Operation	Remark
A	Q	Q_{-1}	M		
0000	1101	0	0111	Initial configuration	
1001	1101	0		A=A-M=A+ M^{2s}	First Cycle
1100	1110	1		Shift A, Q, Q_{-1} right	
0011	1110	1		A=A+M	Second Cycle
0001	1111	0		Shift A, Q, Q_{-1} right	
1010	1111	0		A=A-M=A+ M^{2s}	Third Cycle
1101	0111	1		Shift A, Q, Q_{-1} right	
1101	0111	1		No Add/Sub	Fourth Cycle
1110	1011	1		Shift A, Q, Q_{-1} right	

Result = Contains of AQ register = 1110 1011= -(2's of 1110 1011) =-(00010101) = -21

4. **Example of Booth's Algorithm (-7 *-3 = +21)**
Here Multiplicand, M=7= 1001 and Multiplier, Q=-3=1101
2's complement of M, M^{2s}= 0111

Register	Multiplier Register		Multiplicand Register	Operation	Remark
A	Q	Q_{-1}	M		
0000	1101	0	1001	Initial configuration	
0111	1101	0		A=A-M=A+ M^{2s}	First Cycle
0011	1110	1		Shift A, Q, Q_{-1} right	
1100	1110	1		A=A+M	Second Cycle
1110	0111	0		Shift A, Q, Q_{-1} right	
0101	0111	0		A=A-M=A+ M^{2s}	Third Cycle
0010	1011	1		Shift A, Q, Q_{-1} right	
0010	1011	1		No Add/Sub	Fourth Cycle
0001	0101	1		Shift A, Q, Q_{-1} right	

Result = Contains of AQ register = 0001 0101 = 21

5. Example of Booth's Algorithm (8 *11 = +88)

Here Multiplicand, M= 01000 and Multiplier, Q=11=01011

2's complement of M, M^{2s}= 11000

Register	Multiplier Register		Multiplicand Register	Operation	Remark
A	Q	Q_{-1}	M		
0000	01011	0	01000	Initial configuration	
11000	01011	0		A=A-M=A+ M^{2s}	First Cycle
11100	00101	1		Shift A, Q, Q_{-1} right	
11100	00101	1		No add/Sub	Second Cycle
11110	00010	1		Shift A, Q, Q_{-1} right	
00110	00010	1		A=A+M	Third Cycle
00011	00001	0		Shift A, Q, Q_{-1} right	
11011	00001	0		A=A-M=A+ M^{2s}	Fourth Cycle
11101	10000	1		Shift A, Q, Q_{-1} right	
00101	10000	1		A=A+M	Fifth Cycle
00010	11000	0		Shift A, Q, Q_{-1} right	

Result = Contains of AQ register =00010 11000 = 64+16+8=88

4 Fast Multiplication (Bit-pair Recording of Multipliers)

- To speed-up the multiplication process in Booth's algorithm a techniques called Bit-Pair recording is used. It is also called modified Booth's algorithm.
- It halves the maximum number of summands.
- In these techniques, Booth-recorded multiplier bits are grouped in pairs. Then each pair is represented by its equivalent single bit multiplier reducing total number of multiplier bits to half.
- **Group the booth-recoded multiplier bits in pairs- we can observe the following things:-**

The pair (+1 −1) is equivalent to the pair (0 +1) i.e. instead of adding −1 times the multiplicand M at shift position i to +1 times M at position i +1, the same result is obtained by adding +1 times M at position i.

Similarly

(+1 0) is equivalent to (0 +2)

(−1 0) is equivalent to (0 −2)

(−1 +1) is equivalent to (0 −1) and

(+1 −1) is equivalent to (0 +1) as stated above.

Thus if the booth recoded multiplier is examined two-bits at a time, starting from the right, it can be rewritten in a form that requires at most one version of the multiplicand to be added to the partial product.

Table-1

Multiplier Bits (b_{i+1}, b_i, b_{i-1})	Booth Pair (b_{i+1}, b_i)	Recorded Bit-Pir (b_{i+1}, b_i)	Equivalent to single bit (b_i)
000 or 111	(0, 0)	(0, 0)	0
001	(0, +1)	(0, +1)	+1
110	(0, -1)	(0, -1)	-1
011	(+1, 0)	(0, +2)	+2
100	(-1, 0)	(0, -2)	-2
010	(+1, -1)	(0,+1)	+1
101	(-1, +1)	(0, -1)	-1
	(+1, +1)	-	-
	(-1, -1)	-	-

- Also can be derived directly from multiplier bits taking three bits at a time.

Table-2

Multiplier Bits			Multiplicand selected at position i	Remarks
b_{i+1}	b_i	b_{i-1}	b_i	
0	0	0	0 x M	
0	0	1	+1 x M	
0	1	0	+1 x M	
0	1	1	+2 x M	
1	0	0	-2 x M	
1	0	1	-1 x M	
1	1	0	-1 x M	
1	1	1	0 x M	

Example-1: Referring to Table-1

Implied 0 to the right of LSB

Given Multiplier	1	1	1	0	1	0	(0)
Booth recording	0	0	-1	+1	-1	0	
Bit-pair recording	0	0	0	-1	0	-2	
0ne bit equivalent		**0**		**-1**		**-2**	

Example-2: Referring to Table-2

Implied 0 to the right of LSB

Given Multiplier	1	1	1	0	1	0	(0)
0ne bit equivalent		**0**		**-1**		**-2**	

- **Example-3**
  Multiplier
  0 1 1 1 0 0 0 0
  After Booth's conversion
  +10 0 –1 0 0 0 0
  After pairing
  0+2 0 –1 0 0 0 0
- **Example-4**
  Multiplier
  0 1 1 1 0 1 1 0
  After Booth's conversion
  +1 0 0 –1 +1 0 –1 0
  After pairing
  0 + 2 0 0 –1 0 –1 0
- **Example-5**
  Multiplier
  0 0 0 0 0 1 1 1
  After Booth's conversion
  0 0 0 0 +1 0 0 –1
  After pairing

0 0 0 0 0 +2 0 –1

- **Example-6**
  Multiplier
  0 1 0 1 0 1 0 1

After Booth's conversion
+1–1+1–1+1–1+1–1
After pairing
0 +1 0 +1 0 +1 0 +1

Question-1:
Perform the following multiplication by Bit-Pair recording of multiplier
+13 x -6

Solution:
Here Multiplier, M=+13=01101
Multiplier, Q=-6=11010
=0 -1 +1 -1 0 (Booth Encoding)
=0 -1 -2 (Bit Pair encoding)

						0	1	1	0	1	(+13)	Action	Remarks
					x	0		-1		-2	(-6)		
	1	1	1	1	1	0	0	1	1	0		-2 x M	= $M^{2s} << 1$
	1	1	1	1	0	0	1	1				-1 x M	= M^{2s}
	0	0	0	0	0	0						0 x M	= 0
	1	1	1	0	1	1	0	0	1	0			

Sign. bit.

As the sign. of the product is 1, so the product is –ve (is in sign-2's complement form). So the product in decimal is
= - (2's of the product)
= - (2's of 1110110010)
= - (0001001110) = - ($2^6 + 2^3 + 2^2 + 2^1$)
= - (64+8+4+2) = -78 (**correct result**)

−2 indicates that the 2's compliment of the multiplicand should be left shifted one place
−1 indicates that 2's compliment of the multiplicand should be put in the correct position itself.

Question-2:
Perform the following multiplication by Bit-Pair recording of multiplier
A X B where A = 110101 B = 011011

Solution:
Here A=110101 is a –ve number, so in decimal = -(2's of 110101) = -(001011) = -11
B=011011=+27
So the result should be -11 x +27 = -297
Do it yourself by bit-pair method.

- **Advantages**
  Bit-pair recoding of multiplier requires only half the number of summands compared to the normal algorithm.

********Any error found, mail to **anil.swainfcs@kiit.ac.in** or **anilkumarswain@gmail.com********